

ECE 315

Steven Knudsen, PhD, PEng

Lecture 5 :

- More on OS concepts via FreeRTOS
- FreeRTOS Examples



UNIVERSITY
OF ALBERTA

Last Lecture

- Finish Microprocessor/controller architecture review
- Operating Systems
- PetaLinux
- RTOS Introduction
- Assignment #1a and #1b

This Lecture

- More on RTOS concepts via FreeRTOS
- Patterns
- FreeRTOS example showing two patterns

Learning Objectives

- Know and understand the main RTOS scheduling algorithms
- Understand in detail FreeRTOS scheduling algorithms for single-core
- Know of FreeRTOS scheduling algorithms for multi-core/processors
- Understand simple multi-task program operation in FreeRTOS
- Know what programming patterns are and some examples
- Try first FreeRTOS program
- Introduced to how programs can be broken into tasks
- Define pre-emption and when it might happen
- Understand how a queue can be used to communicate between tasks

Key Components of an RTOS

- **Scheduler:** Determines which task runs next
- **Tasks, Processes, Threads:** Independent units of execution
- **Interrupt Handling:** Fast response to external events
- **Inter-task Communication:** Message queues, semaphores, shared memory
- **Timers & Clocks:** For managing time-based events

Diving into FreeRTOS

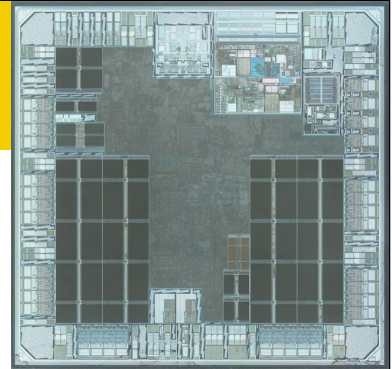
- In this course we will use FreeRTOS
- Open source, and fairly simple, yet robust RTOS
- The principles we explore are common to other RTOSes

Diving into FreeRTOS - Scheduling

- Supports scheduling for
 1. Single core (processor)
 2. Asymmetric multi-core
 3. Symmetric multi-core
- Let's consider configurations 2 and 3

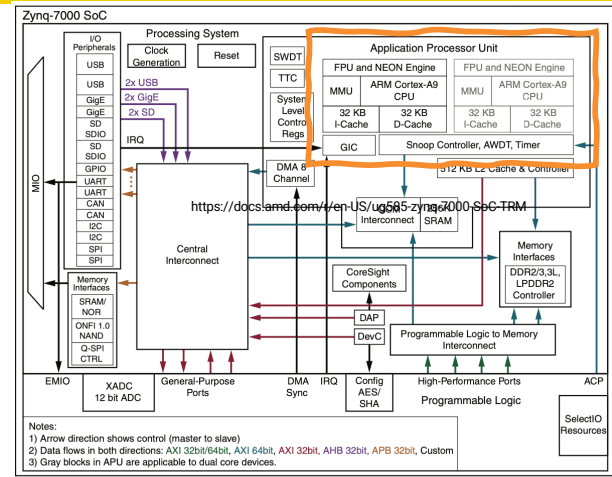
FreeRTOS – AMP Scheduling

- An asymmetric multi-core (multi-processor) configuration
 - Cores or processors are not necessarily different from each other
 - E.g., RPi Pico has dual Cortex-M0+ processors
- Each core/processor runs its own FreeRTOS instance
 - Scheduling is single core
- If cores communicate, they share memory or a channel



FreeRTOS – SMP Scheduling

- A symmetric multi-core (multi-processor) configuration
- Only one FreeRTOS instance schedules tasks for all the other cores
 - All the cores/processors **have to be the same**
- With multiple cores, it is possible to have multiple different priority tasks executing at the same time



FreeRTOS – Single-core Scheduling

- Default is a fixed-priority preemptive scheduling policy, with round-robin time-slicing of equal priority tasks

FreeRTOS – Single-core Scheduling

- Default is a fixed-priority preemptive scheduling policy, with round-robin time-slicing of equal priority tasks

That is a mouthful!

FreeRTOS – Single-core Scheduling

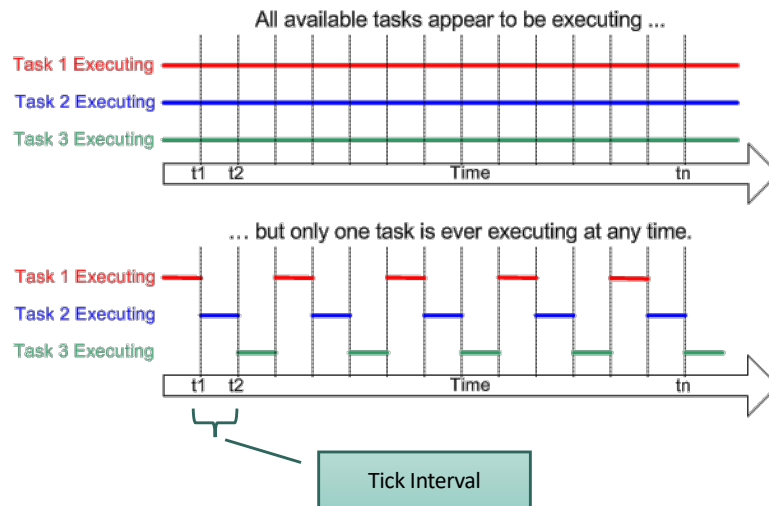
- Default is a fixed-priority preemptive scheduling policy, with round-robin time-slicing of equal priority tasks
- "Fixed-priority" → a task's priority will never be changed except to possibly boost it
- "Preemptive" → the highest priority tasks are always run
 - If a new, higher priority task appears, a lower priority one will be paused
 - The lower priority task is "preempted"
- "Round-Robin" → tasks of the same priority take turns running

FreeRTOS – Single-core Scheduling

- Default is a fixed-priority preemptive scheduling policy, with round-robin time-slicing of equal priority tasks
- "Time Sliced" → scheduler switches between tasks of the same priority each "tick interrupt"
- "Tick Interrupt" → a periodic interrupt used by the RTOS to measure time

FreeRTOS – Time Slicing

<https://www.freertos.org/Documentation/01-FreeRTOS-quick-start/01-Beginners-guide/01-RTOS-fundamentals>



Potential Issue

- A fixed-priority, preemptive scheduler runs the risk that a high priority task “starves” lower priority tasks if it never ends
 - Or never enters a “**Blocked**” or “**Suspended**” state – we will look at that soon
- One solution is to create tasks that are event-driven
 - When the task reaches the point where something needs to happen (an event), it can (should) choose to **Block** itself, relinquish the CPU resource
 - When the event happens, the task is unblocked and resumes
 - This may result in another task being preempted
 - While the higher priority task is blocked, lower priority tasks may run

This is one example of a resource not being given up and impacting all other processes. A mechanism to prevent this or get around it is needed.

FreeRTOS – Configuring the Scheduler

- Preemption and Time Slicing can be turned on or off when the RTOS is compiled
 - Done by setting constants in a header file
- What might happen if Preemption is disabled?
- What might happen if Time Slicing is disabled?

An example showing tasks and patterns

- In this example we create two different kind of tasks
 - Common patterns for real-time programming

Producer-Consumer

- In real-time programming, the producer-consumer pattern is a concurrency design pattern used to coordinate two or more tasks (or threads) that operate at different rates — one producing data and another consuming it — while maintaining deterministic timing and avoiding data loss or race conditions.

An example showing tasks and patterns cont'd

Timed

- A task that does something periodically according to a time schedule
- E.g., Every 60 seconds check the status of a sensor or the operation of a machine
- While the task is idle, it can be paused and
 - the computer resources given to some other task, or
 - one or more parts of the processor can be put into low-power mode (sleep)
- Nordic Semiconductor's Bluetooth chips are great at sleep modes

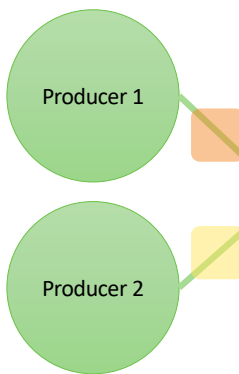
Producer-Consumer

The ***producer-consumer pattern*** (also called the bounded-buffer problem) is a synchronization model where:

- The producer generates data items (e.g., sensor readings, messages, or computation results).
- The consumer processes or uses those data items (e.g., logging, control output, or further computation).
- A shared buffer (queue) is used to temporarily store produced data until the consumer is ready to process it.

Producer-Consumer

Producer Tasks/Threads



Tail

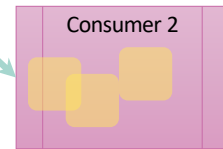
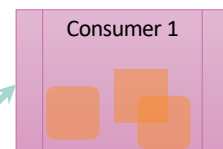
Blocking Queue

Head



Producer-Consumer
Specific data

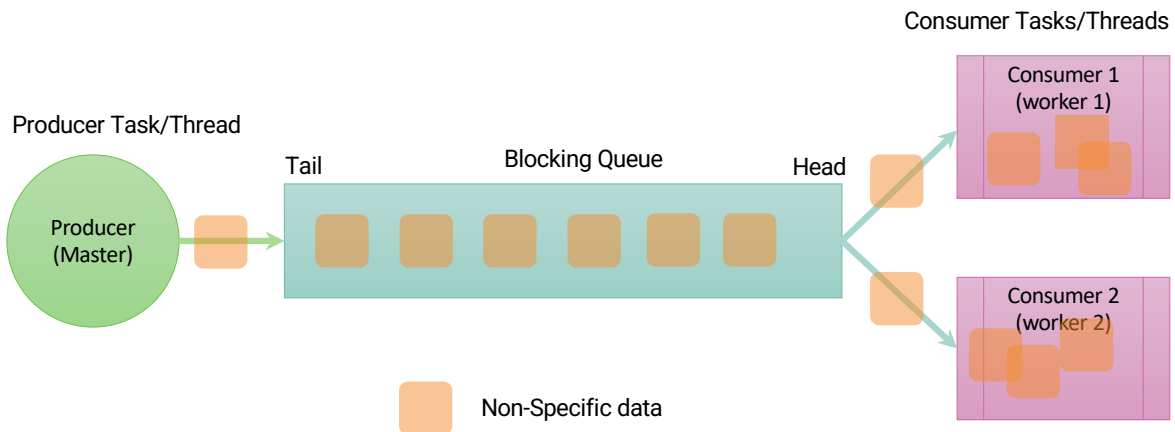
Consumer Tasks/Threads



Adapted from <https://jenkov.com/tutorials/java-concurrency/producer-consumer.html>

20

Producer-Consumer aka Master-Worker



This is known as a programming pattern.

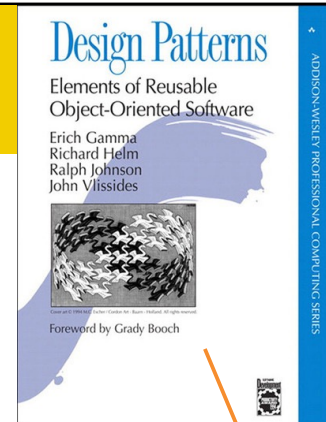
A pattern is a way of doing things that happens often, often enough that it applies to many problems. No point reinventing it...

An Aside - Patterns



Design Pattern

- A proven template or method for solving a class of problems
- Common in programming
 1. **Reusable** – A pattern provides a solution that can be applied to multiple software projects.
 2. **Language-independent** – It describes relationships and interactions between objects or components, not specific syntax.
 3. **Proven** – It's derived from experience and has been tested by many developers over time.
 4. **Structured** – Usually includes a name, the problem it solves, the context in which it applies, and the solution's structure.



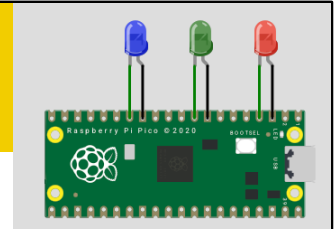
Gang of Four(!)

Aside over...



ECE 315 Lec 5 FreeRTOS Example

<https://wokwi.com/projects/446088208076774401>



- Create a producer-consumer task pair to blink the Pico onboard LED
- Create 3 timed tasks to blink discrete LEDs (R, B, G)
- Notice that all the FreeRTOS source files are included in the project.
 - Easiest way to run FreeRTOS on Wokwi
 - For real hardware, we would use a library of pre-compiled FreeRTOS

ECE 315 Lec 5 FreeRTOS Example

```
/**
 * ECE 315 Computer Interfacing
 *
 * @file main.c
 * @details A simple example to illustrate tasks
 * @author Steven Knudsen
 * @copyright 2025, University of Alberta
 * @version 1.0
 * @licence MIT
 */
#include "main.h"

/*
 * GLOBALS
 */

// This is the inter-task queue
volatile QueueHandle_t queue = NULL;

// Set a delay time of exactly 500ms
const TickType_t ms_delay = 500 / portTICK_PERIOD_MS;

// References to the tasks
TaskHandle_t red_task_handle = NULL;
TaskHandle_t green_task_handle = NULL;
TaskHandle_t blue_task_handle = NULL;

TaskHandle_t pico_producer_task_handle = NULL;
TaskHandle_t pico_consumer_task_handle = NULL;

TaskHandle_t blocking_task_handle = NULL;
```

ECE 315 Lec 5 FreeRTOS Example cont'd

```
/*  
 * FUNCTIONS  
 */  
  
/**  
 * @brief Return number of milliseconds since program start  
 */  
uint64_t get_ms_freertos(){  
    return xTaskGetTickCount()/portTICK_PERIOD_MS;  
}  
  
/**  
 * @brief Block to show preemption  
 */  
void blocking_task(void* unused_arg) {  
    while (true);  
}
```

ECE 315 Lec 5 FreeRTOS Example cont'd

```
/**
 * @brief Repeatedly flash the Pico's built-in LED.
 */
void led_task_consumer(void* unused_arg) {

    // This variable will take a copy of the value
    // added to the FreeRTOS xQueue
    uint8_t passed_value_buffer = 0;

    // Configure the Pico's on-board LED
    gpio_init(PICO_DEFAULT_LED_PIN);
    gpio_set_dir(PICO_DEFAULT_LED_PIN, GPIO_OUT);
```

```
    while (true) {
        // Check for an item in the FreeRTOS xQueue
        if (xQueueReceive(queue, &passed_value_buffer, portMAX_DELAY) == pdPASS) {
            // Received a value so flash the GPIO LED accordingly
            // (NOT the sent value)

            printf("%-9llms consume %d\n", get_ms_freertos(), passed_value_buffer);
            gpio_put(PICO_DEFAULT_LED_PIN, passed_value_buffer == 1 ? 0 : 1);
        }
    }
}
```

ECE 315 Lec 5 FreeRTOS Example cont'd

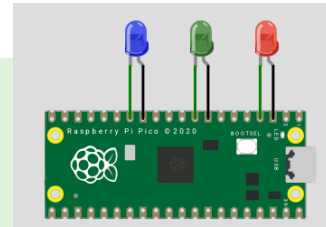
```
/**
 * @brief Repeatedly flash the Pico's built-in LED.
 */
void led_task_producer(void* unused_arg) {
    // Store the Pico LED state
    uint8_t pico_led_state = 0;

    while (true) {
        // Add the LED state to the FreeRTOS xQUEUE
        pico_led_state = !pico_led_state; // toggle LED state
        printf("%-9llms produce %d\n", get_ms_freertos(), pico_led_state);
        xQueueSendToBack(queue, &pico_led_state, 0);
        vTaskDelay(ms_delay);
    }
}
```

ECE 315 Lec 5 FreeRTOS Example cont'd

```
/*
 * LED task parameters struct
 */
struct led_config {
    uint8_t led_pin;
    TickType_t ms_delay;
    char* name;
};

struct led_config red_config = {RED_LED_PIN, 500 / portTICK_PERIOD_MS, "RED"};
struct led_config green_config = {GREEN_LED_PIN, 1000 / portTICK_PERIOD_MS, "GREEN"};
struct led_config blue_config = {BLUE_LED_PIN, 2000 / portTICK_PERIOD_MS, "BLUE"};
```



ECE 315 Lec 5 FreeRTOS Example cont'd

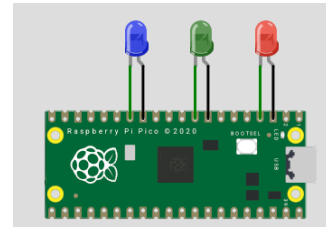
```
/**
 * @brief Repeatedly flash an LED connected to the specified GPIO
 * based on the value passed via the inter-task queue.
 */
```

```
void led_task_gpio(void* params) {
```

```
    struct led_config cfg = *((struct led_config*) params);
    uint8_t led_state = 1;
```

```
    // Configure the GPIO LED
    gpio_init(cfg.led_pin);
    gpio_set_dir(cfg.led_pin, GPIO_OUT);
```

```
    while (true) {
        // Toggle the LED
        led_state = !led_state;
        printf("%-9llums %s %d\n", get_ms_freertos(), cfg.name, led_state);
        gpio_put(cfg.led_pin, led_state);
        vTaskDelay(cfg.ms_delay);
    }
}
```



ECE 315 Lec 5 FreeRTOS Example cont'd

```
/*
 * RUNTIME START
 */
int main() {

    // Enable STDIO
    stdio_init_all();

    // Set up two tasks
    // Store handles referencing the tasks; get return values
    // NOTE Arg 3 is the stack depth -- in words, not bytes
    BaseType_t prod_status = xTaskCreate(led_task_producer,
                                         "PICO_PRODUCER_TASK",
                                         1024,
                                         NULL,
                                         1,
                                         &pico_producer_task_handle);

    BaseType_t con_status = xTaskCreate(led_task_consumer,
                                         "PICO_CONSUMER_TASK",
                                         1024,
                                         NULL,
                                         1,
                                         &pico_consumer_task_handle);
}
```



The diagram illustrates the flow of the code. Arrows point from the `xTaskCreate` function calls to the task names (`"PICO_PRODUCER_TASK"` and `"PICO_CONSUMER_TASK"`) and the task handles (`&pico_producer_task_handle` and `&pico_consumer_task_handle`).

ECE 315 Lec 5 FreeRTOS Example cont'd

```
BaseType_t red_status = xTaskCreate(led_task_gpio,
                                   "RED_LED_TASK",
                                   1024,
                                   &red_config,
                                   1,
                                   &red_task_handle);

BaseType_t green_status = xTaskCreate(led_task_gpio,
                                      "GREEN_LED_TASK",
                                      1024,
                                      &green_config,
                                      1,
                                      &green_task_handle);

BaseType_t blue_status = xTaskCreate(led_task_gpio,
                                     "BLUE_LED_TASK",
                                     1024,
                                     &blue_config,
                                     1,
                                     &blue_task_handle);
```

ECE 315 Lec 5 FreeRTOS Example cont'd

```
#if WITH_BLOCKING
    BaseType_t block_status = xTaskCreate(blocking_task,
                                           "BLOCKING_TASK",
                                           1024,
                                           NULL,
                                           1,
                                           &blocking_task_handle);
#else
    BaseType_t block_status = pdPASS;
#endif
```

ECE 315 Lec 5 FreeRTOS Example cont'd

```
// Set up the event queue
queue = xQueueCreate(4, sizeof(uint8_t));

// Start the FreeRTOS scheduler
// Only proceed with valid tasks
if (prod_status == pdPASS || con_status == pdPASS || red_status == pdPASS || \
    green_status == pdPASS || blue_status == pdPASS || block_status == pdPASS) {
    vTaskStartScheduler();
}

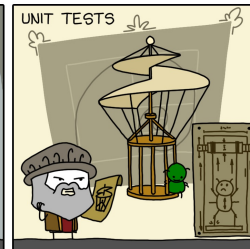
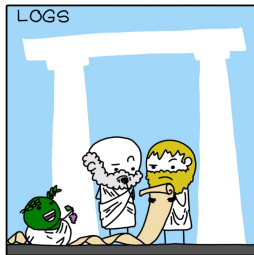
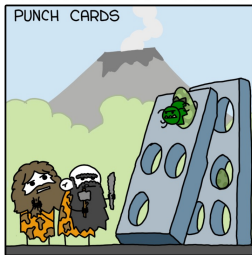
// We should never get here, but just in case...
while(true) {
    // NOP
}
```

Next Lecture

- Embedded Software
- Design Thoughts
- Real-Time Specifications and Constraints

HISTORY OF DEBUGGING

MONKEYUSER.COM



Questions

